HW05 - Summary of Object Oriented Modeling and UML

L.05.01 - Object-Oriented Good Practices

Concept modeling

Conceptual modeling helps developers understand real-world problems before creating software. It's useful for analyzing requirements, organizing ideas, and making design decisions.

Main Principles

- Use UML to represent software systems visually.
  Organize software into independents layers.
  Reduce unnecessary dependencies between components.
  Prefer clean abstractions and meaningful names.
  Apply testing to ensure software quality.

Importance: Good modeling practices improve communication, maintainability, and system organization.

L.05.07 - Introduction to UML

UML Definition:

The Unified Modeling Language (UML) is a standardized visual language for describing object-oriented systems.

Main Features

UML is flexible, expressive, and suitable for different software development processes. It allows developers to create models progressively.

Types of Diagrams

Structural Diagrams = Describe the organization of a system.
Behavioral Diagrams = Represent activities, states and interactions.
Use Case Diagrams = Identify user goals.
Class Diagrams = Represent classes and their connections.

MODELING

principles:

Analysis: Understands the real-world problem.

Specification: Define the logical structure.

Implementation: Describe how software components are built.

1.05.08 - Basic Use Cases

Actors and use cases

An actor is a person, external system, or device that interacts with an application. A use case represents a goal that an actor wants to achieve.

Use Case Diagram Elements

Actor: External participant

Use Case: User objective

Subject: System being modeled.

Relationship: Connection between elements.

Types of relationship

- <<include>>: One use case requires another.
- <<extend>>: Adds behavior in special situations
Generalization: Represents a specialized actor or use case.

Important

Difference

A use case focuses on what users need, while system interactions explain how users accomplish their goals

1.05.09 Objects, Classes and Basic Class Diagrams

Objects and Classes

An object represents an identifiable element with its own properties and behavior.
A class group similar objects and defines their shared characteristics

Escaneado con CamScanner

UML Class Representation

A class is commonly represented by a rectangle divided into three sections
1) Class Name
2) Attributes
3) Operations

Relationships Between Classes

- Association = General connection between classes
- Inheritance: A specialized class shares characteristics with a general class.
- Aggregation: A whole contains parts that can exist independently
- Composition: Parts depend on the whole
- Dependency One class temporarily uses another.

Multiplicity

Multiplicity defines the number of instances participating in a relationship, such as 1, 0..1, 0..*, and 1..*.
Class diagrams help developers understand the static structure of an application.

1 05.10 - UML Quick Reference Guide

P U R P O S E

The UML Quick Reference Guide explains common modeling elements and recommendations for creating understandable diagrams.

Important UML Diagrams

- Use Case Diagram: Represents system goals and external actors
- Class Diagram: Displays classes and their structural relationships.
- Activity Diagram: Shows processes and decision paths
- Sequence Diagram: Represents messages exchanged between objects.
- State Diagram: Describes changes in an object's state.

Escaneado con CamScanner

Recommended PRACTICES

- Use simple and descriptive names, represent relationships correctly.
- Avoid overcrowded diagrams.
- Minimize crossing lines.
- Select diagrams according to the modeling objective.
- Keep models focused on the problem being analyzed.

Conclusion

UML is an important tool in object-oriented programming because it allows developers to understand, visualize and communicate software requirements and system structures before implementation.

Escaneado con CamScanner